

// SISTEMAS EMBEBIDOS (3.3.127) • UADE

# TP Integrador — Variante 3

Caja de comandos física para un robot Unitree Go2 / G1

// INTEGRANTES

**Borda Rojas • Castiglioni**

Profesor **Dr. Ing. Leonardo J. Amet**

Curso **Miércoles Noche** Fecha **15/07/2026**

pill → bridge ROS 2

@0E:EVT:cmd\_deploy:0C

@0F:STS:mode=DEPLOY:17

@0D:DAT:param=094:4A

→ ACK:ok

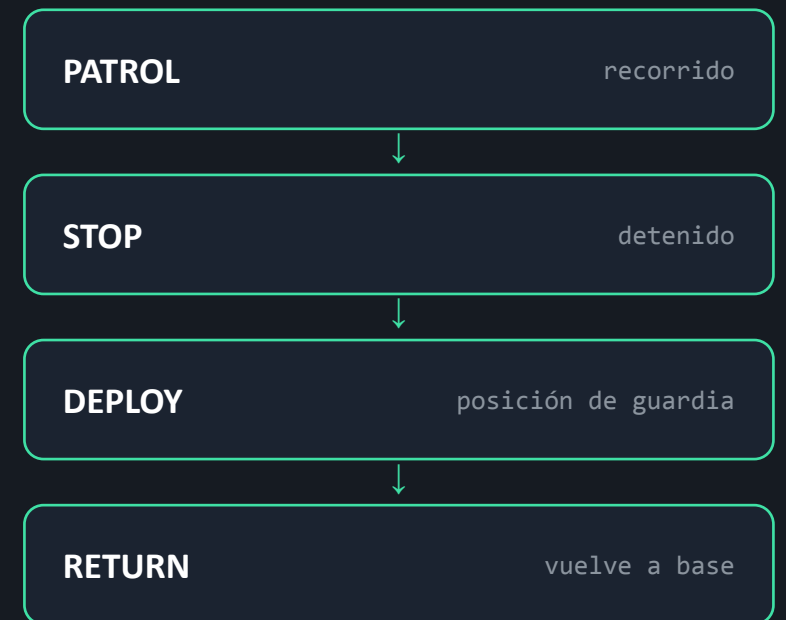
# Objetivo y qué hace la Blue Pill

La Blue Pill no controla el robot: es su **interfaz física**. Se comunica por UART con un bridge ROS 2 de la cátedra, que traduce las tramas en tópicos, comandos y servicios que el robot entiende.

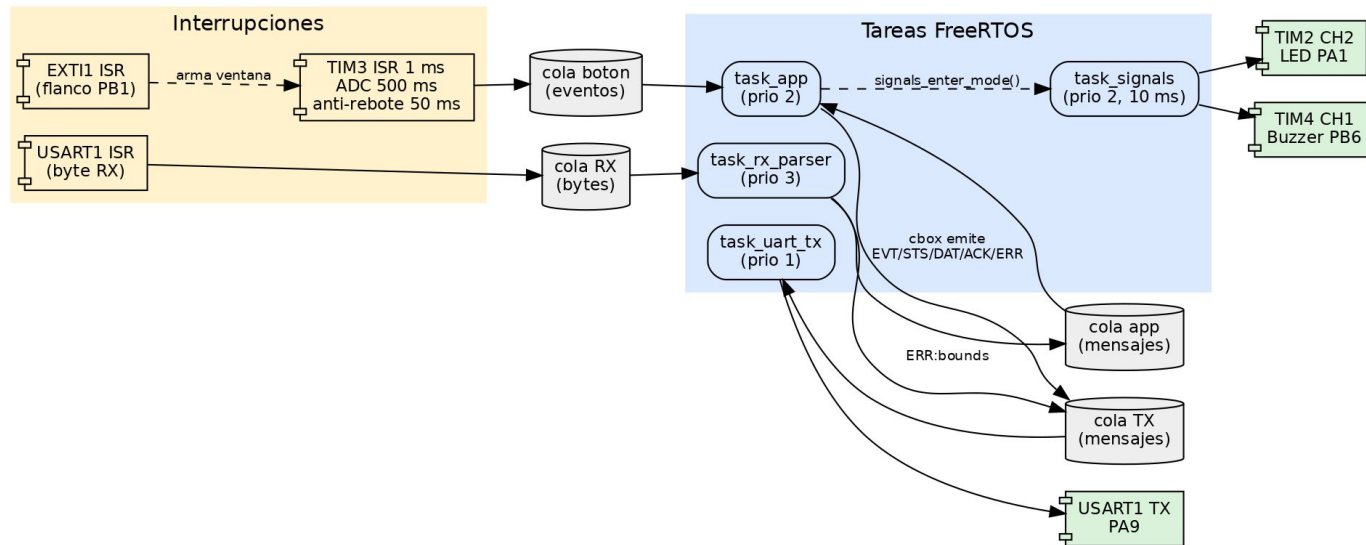
- ▶ **Pulsador (EXTI1)**: cada pulsación válida cicla al modo siguiente del robot.
- ▶ **Potenciómetro (ADC)**: define el parámetro auxiliar 0–255 que viaja con cada comando.
- ▶ **LED + buzzer** : señalizan el modo activo con patrón y código de pitidos.
- ▶ **Protocolo serie**: genera tramas válidas y consume las respuestas del bridge.
- ▶ **Arranque seguro**: la caja inicia en STOP; no mueve nada hasta la primera pulsación.

De 5 variantes cerradas, elegimos la **Variante 3 · Caja de comandos física**.

## Ciclo de modos



# De la interrupción al robot



Tareas FreeRTOS, ISR y colas. Las ISR sólo encolan; el trabajo corre en tareas.

4

tareas

4

colas

3

fuentes ISR

1 ms

base de tiempo

- ▶ **Interrupciones mínimas:** USART1 encola bytes, EXTI1 arma el anti-rebote, TIM3 da la base de 1 ms.
- ▶ **Colas FreeRTOS:** desacoplan recepción, lógica y transmisión entre contextos.
- ▶ **Lógica pura:** `command_box` y `signals` no tocan registros → testeables en PC.
- ▶ **Robustez:** `configASSERT` valida colas y tareas; hooks de heap y stack overflow.

# Trama serie y tipos de mensaje

formato de trama • UART 115200 8N1

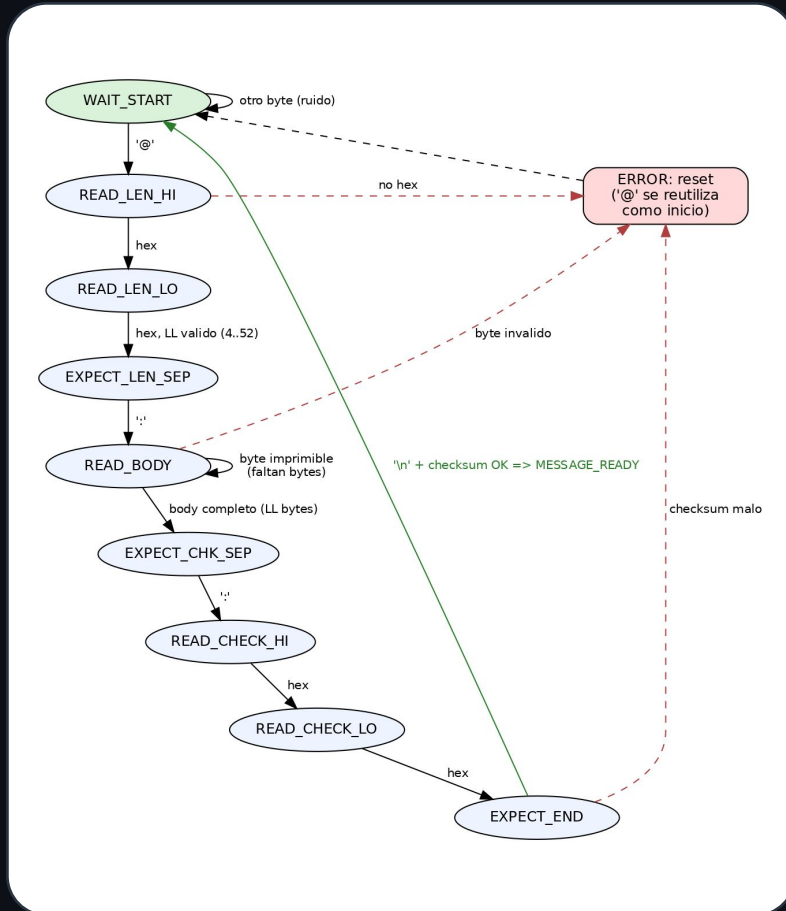
@LL:TTT:PAYLOAD:CC\n

LL long. hex del cuerpo • CC = XOR de LL:TTT:PAYLOAD

- ▶ **Payload ASCII** imprimible, ≤ 48 bytes, sin '@'.
- ▶ **Checksum XOR** byte a byte, en 2 dígitos hexadecimales.
- ▶ Ejemplo: cuerpo `EVT:cmd_deploy` (14 bytes) → LL=0E, CC=0C.
- ▶ Trama final: `@0E:EVT:cmd_deploy:0C`

| Tipo | Dirección     | Uso             |
|------|---------------|-----------------|
| CMD  | Bridge→Pill   | comandos        |
| DAT  | Pill→Bridge   | datos ADC/param |
| EVT  | Pill→Bridge   | eventos         |
| STS  | Pill→Bridge   | estado/modo     |
| ACK  | bidireccional | confirmación    |
| ERR  | Pill→Bridge   | bounds/timeout  |

# Parser incremental (FSM de 9 estados)



Máquina de estados del parser.

- **Byte a byte:** la UART es un flujo sin alineación.
- **Valida** longitud, tipo, caracteres y checksum.
- **Resincroniza:** descarta la trama rota y reutiliza '@'.
- **Sin bloqueo:** no espera un '\n' que puede no llegar.

recorrido FSM • @0C:DAT:adc=2048:77

| byte | estado                       | → nuevo         |
|------|------------------------------|-----------------|
| @    | WAIT_START                   | → LEN_HI        |
| 0    | LEN_HI                       | → LEN_LO        |
| C    | LEN_LO                       | → LEN_SEP       |
| :    | LEN_SEP                      | → BODY          |
| D..8 | BODY (12 bytes DAT:adc=2048) |                 |
| :    | CHK_SEP                      | → CHK_HI        |
| 7 7  | CHK_HI/LO                    | → EXPECT_END    |
| \n   | EXPECT_END                   | → MESSAGE_READY |

@0A:CMD:ping:52 → LL=10 pero el cuerpo mide 8 → rechaza y resync.

# Lazo de eco y cálculo de checksum

● ● ● evidencia/etapa1\_tests\_protocolo.txt

```
TX @0F:CMD:mode=DEPLOY:09 → @06:ACK:ok:4B ok
TX @0E:CMD:mode=ESTOP:5C → @06:ACK:ok:4B ok
TX @06:STS:OK:56 → @06:ACK:ok:4B ok
TX @0A:EVT:e_stop:14 → @06:ACK:ok:4B ok
TX @0E:EVT:cmd_deploy:0D → @0A:ERR:bounds:35 checksum
TX @0A:CMD:ping:52 → @0A:ERR:bounds:35 LL malo
```

Checksum de EVT:cmd\_deploy

```
30^45^3A^45^56^54^3A
^63^6D^64^5F^64^65^70
^6C^6F^79

= 0x0C
```

@0E:EVT:cmd\_deploy:0C

Trama válida → **ACK:ok** · trama inválida → **ERR:bounds**. El checksum y la longitud LL detectan la corrupción; el parser descarta y se resincroniza sin perder la siguiente trama.

# Núcleo de periféricos y timers

| Periférico | Pin      | Función                        | Valor  |
|------------|----------|--------------------------------|--------|
| USART1     | PA9/PA10 | Enlace serie con el bridge     | 115200 |
| ADC1       | PA0      | Potenciómetro, conv. c/500 ms  | 12 MHz |
| TIM2       | PA1      | PWM del LED de modo (CH2)      | 1 kHz  |
| TIM4       | PB6      | Tono del buzzer pasivo (CH1)   | 2 kHz  |
| TIM3       | —        | Base: uptime, ADC, anti-rebote | 1 ms   |
| EXTI1      | PB1      | Pulsador, pull-down externo    | 50 ms  |

ADC: prescaler /6, muestreo 55,5 ciclos y calibración al arranque · pines de PWM en modo alternativo push-pull.

## Cálculos clave

**PWM LED (TIM2)**  
 $f = 72M / (71+1) / (999+1)$   
 $= 1 \text{ kHz}$   
 $CCR2 = duty \times 10$

**Buzzer (TIM4)**  
 $ARR=499 \rightarrow 2 \text{ kHz}, 50\%$

**Base (TIM3)**  
 $PSC=7199, ARR=9 \rightarrow 1 \text{ ms}$

**Mapeo**  
 $param = adc \times 255 / 4095$

# Ciclo de modos y señalización



| Modo   | Tramas emitidas                  | LED            | Buzzer    |
|--------|----------------------------------|----------------|-----------|
| PATROL | EVT:cmd_patrol · STS:mode=PATROL | destello 1 Hz  | 1 pitido  |
| STOP   | EVT:cmd_stop · STS:mode=STOP     | fijo           | 2 pitidos |
| DEPLOY | EVT:cmd_deploy · STS:mode=DEPLOY | destello 4 Hz  | 3 pitidos |
| RETURN | EVT:cmd_return · STS:mode=STOP   | doble destello | 4 pitidos |

● ● ● RETURN informa mode=STOP: el robot vuelve a base y queda detenido

cada transición también envía DAT:param=NNN · ancho fijo con ceros: sprintf("param=%03u") → param=007

# ACK, reintentos y timeout

500 ms

espera de ACK:ok por evento

5

reenvíos del mismo EVT

ERR:timeout

al agotar los reintentos

- ▶ **Una transacción a la vez:** el protocolo no lleva ID; no hay dos EVT en vuelo.
- ▶ **Reenvío automático:** si no llega ACK:ok en 500 ms, se reenvía el evento.
- ▶ **Degradación segura:** tras 5 intentos emite ERR:timeout y la caja sigue.
- ▶ **Recuperación sola:** si el bridge vuelve durante los reenvíos, continúa.

● ● ● sesión simulada · reintentos y recuperación

```
[ 2.000] EVT:cmd_deploy + STS + DAT
[ 2.040] ACK:ok ✓
[10.075] EVT:cmd_patrol (bridge caído)
[10.575] reenvío 1
[11.075] reenvío 2
[11.295] ACK:ok (bridge vuelve) ✓
[18.333] EVT:cmd_deploy (sin bridge)
[18.8..20.8] 5 reenvíos c/500 ms
[21.333] ERR:timeout ×
```

# Reparto en tareas FreeRTOS

| Tarea                       | Rol                            | Prio | Stack |
|-----------------------------|--------------------------------|------|-------|
| <code>task_rx_parser</code> | Corre la FSM byte a byte       | 3    | 256   |
| <code>task_app</code>       | Botón → modo; ACK; reintentos  | 2    | 256   |
| <code>task_signals</code>   | Patrones LED y pitidos c/10 ms | 2    | 128   |
| <code>task_uart_tx</code>   | Serializa y transmite tramas   | 1    | 256   |

flujo de datos

USART1 ISR

→ cola RX → parser → app

EXTI1 / TIM3 ISR

→ cola botón → app

`task_app` (callback)

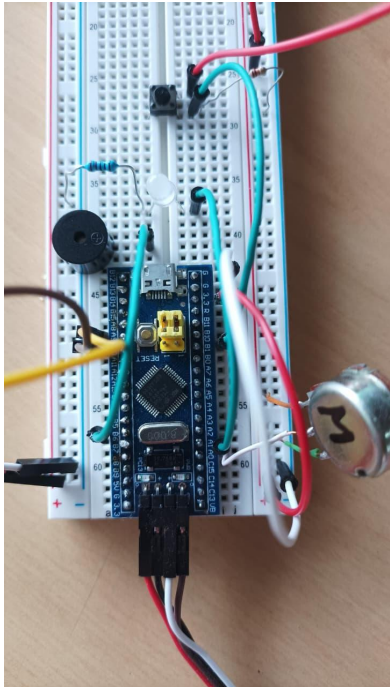
→ cola TX → UART

`task_signals`

→ LED (TIM2) · buzzer (TIM4)

`configASSERT` + `heap` +  
`stack-overflow` hook

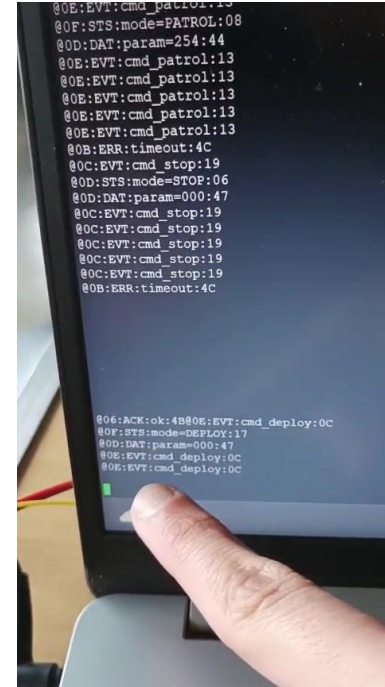
# Montaje y prueba sobre hardware



Montaje en protoboard.



Operación: ADC, EXTI y señalización.



ACK:ok confirmado por UART.

Blue Pill · pulsador con pull-down · LED con resistor de 220  $\Omega$  · buzzer pasivo · potenciómetro de 10 k $\Omega$  · adaptador USB-UART. **Verificado con PuTTY a 115200 8N1.**

# Integración con el robot

## ✓ Validado

- ▶ Firmware, UART y parser.
- ▶ ADC, PWM LED/buzzer, EXTI.
- ▶ ACK, reintentos y timeout.
- ▶ Enlace UART con PuTTY.
- ▶ 67 tests + CI en verde.

## 🕒 Pendiente en laboratorio

- ▶ Tópicos ROS 2 (topic list/echo).
- ▶ Latencia evento → robot.

# Pruebas y evidencia reproducible

67

tests de host (make run)

CI

make test + build ARM

bridge\_sim

sesión simulada con reintentos

- ▶ **Cobertura:** protocolo, parser, lógica de la variante y motor de señales.
- ▶ **Reproducible:** tests y bridge simulado generan evidencia en PC, sin hardware.
- ▶ **Diagnóstico FreeRTOS:** verifica colas/tareas; detiene ante falta de heap o stack overflow.
- ▶ **Trazabilidad:** cada prueba enlaza enunciado, código y evidencia.

```
tests / CI

make run    → 67 tests OK
make fsm    → recorrido FSM
make sim     → sesión ~25 s

protocolo · parser · variante
· señales · config RTOS

CI: push/PR → test + ARM
```

// CIERRE

# ¡Gracias!

¿Preguntas?

Borda Rojas · Castiglioni · TP Integrador · Variante 3



repositorio

[github.com/jiborjas/TPI-V3](https://github.com/jiborjas/TPI-V3)